Dynamic, Conversation-Driven UI for QuantUI — Feasibility Assessment

Brainstorm capture, 2026-07-03. Context: the current QuantUI is dashboard-centric; the idea under discussion is a more dynamic interface that the LLM changes and manipulates based on the context of the conversation between the user and the LLM.

Verdict

For this codebase specifically, the idea is not just reasonable — it is unusually low-risk for how futuristic it feels. The pattern (generative UI / LLM-composed interfaces) is a year-plus into being a known quantity (Vercel AI SDK generative UI, MCP Apps / MCP-UI), so this would be assembly, not invention.

The spectrum of "dynamic"

Four levels, in ascending ambition:

1. **Chat panel that renders components.** A conversation rail in QuantUI; when the LLM answers "how's INTC looking?", the response streams a directive like `{component: "SignalsPanel", props: {symbol: "INTC"}}` and existing React components (MaxPainChart, PCRatioChart, SignalsTab, the securities DataGrid) render live data. The LLM composes; the components render truth from the API.

2. **Conversation-driven workspace.** The conversation is the control plane for a persistent canvas. "Compare INTC to AMD" mutates the view into a comparison layout; "now just show me the spread pricer" collapses everything else. UI state and conversation state are the same state.

3. **Ambient adaptation.** No chat; the dashboard silently reshapes from inferred context (earnings week → earnings panels surface). Risky: UIs that rearrange themselves without an explicit cause tend to feel haunted rather than smart.

4. **Fully generative rendering.** The LLM emits novel JSX/HTML on the fly. Maximum flexibility, but wrong for a finance tool: numbers should be rendered by audited components bound to real API data, not LLM-authored markup where a hallucinated price looks identical to a real one.

Recommendation: level 1 as the foundation, growing into level 2. The LLM's vocabulary is the existing component library plus layout verbs (show, pin, replace, compare, clear). Level 4 stays off the table; level 3 may emerge later as "suggested views" rather than silent rearrangement.

Why this is tractable here

The typical team attempting "LLM drives the UI" must first build three hard things. QuantCore already has all three:

1. **The tool layer is done.** 50+ service methods across prices, options, fundamentals, sentiment, and microstructure, all one service call deep with clean JSON contracts. This is the actual moat.

2. **The component vocabulary is done.** MaxPainChart, PCRatioChart, SignalsTab, the securities DataGrid, and LivePrice are typed, data-bound components fed by React Query hooks. A "registry" is mostly a manifest file pointing at what exists.

3. **The serving/auth problem is solved.** The Express proxy already injects the app JWT server-side; a `/api/chat` endpoint slots into the same pattern, and IAP already gates who can reach the UI.

The honest cost side

• **Latency is the real UX risk.** A question needing 3–4 tool calls means 10–30 seconds before the view updates. Streaming plus optimistic placeholder shells ("pulling INTC signals…") makes it feel alive, but it will never feel like clicking a dashboard tile. Accept this consciously.

• **The novel engineering is state sync, not rendering.** "Now compare it to AMD" only works if the LLM knows what is on screen. The known fix — send a compact workspace manifest with every turn — is straightforward but needs real design care.

• **Per-query LLM cost.** Cents per interaction at team scale; needs a budget guard, not a blocker.

• **The 20% trap.** The demo works in a weekend; error states, stream hiccups, history persistence, and polish are the long tail. Plan for the tail.

Rough v1 shape

• Chat rail + `/api/chat` (FastAPI SSE, Claude with the existing services as tools — preserving the "one service call deep" architectural rule).

• A registry of ~5 existing components; streamed responses interleave prose and UI directives; directives are validated against the registry before rendering (bad directive → degrade to text, never crash the canvas).

• Skip the persistent canvas initially — render components inline in the conversation first, and promote to a workspace once the pattern proves itself.

• Ballpark effort: backend endpoint ~1 day (services are constructor-injected already), registry

• directive renderer 1–2 days, chat UI ~1 day, then prompt tuning as ongoing seasoning.

One more point in favor

The team already lives this workflow: driving the QuantCore MCP tools conversationally from the terminal is exactly this product minus the visual rendering. The pitch is "give the analysis brain hands on the UI."